

## Lógica de Floyd-Hoare

- Método axiomático para provar que determinados programas são corretos.
- Introduzido por Charles Antony Richard Hoare e Robert Floyd na década de 1960.
- Também utilizado para especificar a semântica (axiomática) de linguagens de programação.
- Por que devemos especificar e verificar código?
  - Para redução de falhas que são introduzidas durante a especificação do sistema e, assim, reduzir o seu custo final;
  - Código bem especificado e verificado é mais fácil de reutilizar, uma vez que temos um claro entendimento do que o sistema deve fazer;
  - Para sistemas críticos (aviões, usinas nucleares, ...) isto se torna muito mais importante, pois se espera que os sistemas possam sofrer auditorias sobre o quanto eles garantem que irão funcionar conforme desejado;
  - A especificação de um sistema auxilia na documentação do mesmo e, feita de uma maneira formal, auxilia no processo de implementação final do sistema;
  - Atualmente empresas enfrentam problemas com sistemas legados, pois os responsáveis pelo seu desenvolvimento não os documentam de uma maneira precisa e, na maioria das vezes, não se encontram mais na empresa;
  - Sistemas de software têm um tempo de vida muitas vezes maior do que as pessoas – o *bug* do milênio (ano 2000) é apenas um exemplo.
- Abordagem que usaremos:
  - ➔ aplicar o método de Floyd-Hoare sobre uma linguagem de programação imperativa simplificada.

[Dados de Entrada] → [Programa] → [Dados de Saída]

## **Triplas de Hoare**

$\boxed{(\{P\}) \ C \ (\{Q\})}$ , onde:

- P e Q são asserções (fórmulas na lógica de predicados de primeira ordem com igualdade);
- P : pré-condição (asserções definidas sobre o estado inicial do programa, que denotam sob quais restrições o programa está preparado para funcionar);
- Q : pós-condição (asserções sobre o estado final do programa, que denotam condições sobre os dados resultantes do programa);
- C : comandos de uma linguagem de programação.

Exemplo:

$(\{x > 4\}) \quad x := x + 1 \quad (\{x > 5\})$

### **Exemplo 1:**

Problema: dados dois valores de entrada sobre o domínio dos números inteiros maiores ou iguais a 10, o programa deverá calcular a soma desses dois valores.

Especificação:

- sejam x o primeiro valor de entrada e y o segundo valor de entrada.
- seja z o valor calculado pelo programa.

Solução (A) - OK

pré-condição:  $(\{x \geq 10\} \wedge \{y \geq 10\})$   
 pós-condição:  $(\{z = x + y\})$

Solução (B) – Exemplo de super-especificação

pré-condição:  $(\{x \geq 10\} \wedge \{y \geq 10\} \wedge \{(x+y) \geq 0\} \text{ desnecessário})$   
 pós-condição:  $(\{z = x + y\} \wedge \{z \geq 20\} \text{ desnecessário})$

Solução (C) – Exemplo de especificação desnecessariamente complexa

pré-condição:  $(\{x \geq 10\} \wedge \{(x+y) \geq (10+x)\} \text{ não é claro})$   
 pós-condição:  $(\{z = x + y\})$

Solução (D) – Exemplo de especificação incompleta

pré-condição:  $(\{x \geq 10\} \text{ ??? falta algo!})$   
 pós-condição:  $(\{z = x + y\})$

### **Exemplo 2:**

Problema: dado um número inteiro, calcular o fatorial do número de forma iterativa (não recursiva).

Especificação:

- x: o número (entrada)

- fat: o resultado do programa

Solução:

precondição:  $(| (x \geq 0) \wedge (\text{temp} = x) |)$

pós-condição:  $(| (\text{fat} = \text{temp}!) |)$

Exemplo de possível programa em Java:

```
int fatorial (int x){  
    int fat = 1;  
    while (x != 1){  
        fat = fat * x;  
        x = x - 1;  
    }  
    return fat;  
}
```

*Note que  $x$  será alterado no programa, portanto reservamos uma cópia lógica em temp.*

### **Exercícios:**

- 1) Formular uma especificação para um programa que, dados dois números naturais, deverá calcular o máximo divisor comum para eles (considere uma função matemática MDC).
- 2) Formular uma especificação para um programa que, dados dois números inteiros de entrada, a soma desses dois números no estado inicial deverá ser idêntica à soma deles no estado final do programa. Note que o programa poderá eventualmente modificar os valores das variáveis de entrada.